

Module Workflow Specification

AI Agent Implementation Guide for ConstructPM — 5 Core Modules

Complete backend schemas, API endpoints, frontend services, and step-by-step implementation instructions.

Modules Covered:

DPR & Progress · Spend & Vendors · Payroll · Financial Ledger · Reports & Exports

Tech Stack: Next.js 16 · NestJS 11 · MongoDB 6 · Redis 7

Date: October 13, 2026

Author: Viren Prajapati

FOR AI AGENT CONSUMPTION

ConstructPM — Module Workflow Specification

For AI Agent: Complete Implementation Guide

> **Purpose:** This file is the single source of truth for an AI agent to analyze, understand, and correctly implement/complete the following five modules. Each section maps the **UI sidebar name** (from the reference app) → **this project's actual codebase** (file paths, schemas, controllers, services, API endpoints, and frontend pages).
>
> **Reference App sidebar names (from screenshots):**
>
> 1. DPR & Progress
> 2. Spend & Vendors
> 3. Payroll
> 4. Financial Ledger
> 5. Reports & Exports
>
> **This project's sidebar names (from `frontend/src/components/layout/Sidebar.tsx`):**
>
> 1. Work Management → maps to **DPR & Progress**
> 2. Expenses (nav.payments at `/expense`) + Credit Logs → maps to **Spend & Vendors**
> 3. Payment Central → maps to **Payroll**
> 4. Running Cost + Credit Logs → maps to **Financial Ledger**
> 5. Reports + Report Center → maps to **Reports & Exports**

GLOBAL CONTEXT

Tech Stack

- **Backend:** NestJS 11 (TypeScript), Mongoose 9 (MongoDB 6 with replica set), BullMQ (Redis task queues)
- **Frontend:** Next.js 16 (App Router), React 19, TypeScript, TailwindCSS 4, Zustand (state), SWR (data fetching)
- **Auth:** JWT (HttpOnly cookie) + RBAC (8-role hierarchy)
- **Multi-Tenancy:** Every document has `organizationId`; enforced via AsyncLocalStorage middleware + Mongoose plugin

Key File Locations

```
backend/src/modules/          → All NestJS modules (controller, service, schema, dto)
frontend/src/app/(dashboard)/ → All Next.js pages (App Router)
frontend/src/services/index.ts → All API service functions (axios wrappers)
frontend/src/components/      → Reusable React components
frontend/src/types/index.ts   → TypeScript type definitions
backend/src/common/constants/index.ts → All enums (roles, statuses, types)
```

API Base

- Backend URL: `http://localhost:5000/api/v1`
- All endpoints require JWT auth via `@UseGuards(JwtAuthGuard, RolesGuard)`
- Admin endpoints require: `@Roles(UserRole.ORGANIZATION_SUPER_ADMIN, UserRole.ADMIN)`

Roles (from `backend/src/common/constants/index.ts`)

```
PLATFORM_SUPER_ADMIN → ORGANIZATION_SUPER_ADMIN → SUPER_ADMIN (legacy) → ADMIN → PROJECT_MANAGER → SITE_ENGINEER → SUPERVISOR → USER (leg
```

MODULE 1: DPR & PROGRESS (Daily Progress Reports)

Reference App UI

- **Sidebar:** "DPR & Progress"
- **Page Title:** "Daily Progress Reports (DPR)"
- **Subtitle:** "Site execution milestones, task completion percentages, and material consumption."
- **Primary Action:** "+ Submit Site DPR" button (top-right, blue)
- **Empty State:** Camera icon + "No DPRs Logged Yet" + "Click 'Submit Site DPR' to submit the first shift log."
- **Submit Form (Modal):**
  - Site Project * (dropdown: Select Project)
  - Report Date * (date picker, default: today DD/MM/YYYY)
  - Weather * (text input, e.g. "Sunny 32°C")
  - Shift Work Summary * (textarea, placeholder: "e.g. Completed foundation raft formwork and rebar tying")
  - Buttons: Cancel | ✈ Submit DPR

Maps To (This Project's Codebase)

Backend Module: `work-management`

```
backend/src/modules/work-management/
├── work-management.controller.ts   → API endpoints
├── work-management.service.ts      → Business logic (17KB)
├── work-management.module.ts       → NestJS module
├── dto/                            → Request validation DTOs
│   ├── create-work-entry.dto.ts
│   ├── update-work-entry.dto.ts
│   ├── query-work-entry.dto.ts
│   ├── create-work-group.dto.ts
│   ├── update-work-group.dto.ts
│   ├── query-work-group.dto.ts
│   ├── create-work-report.dto.ts
│   ├── query-work-report.dto.ts
│   └── index.ts
└── schemas/
    ├── work-entry.schema.ts        → Daily work logs (DPR entries)
    ├── work-group.schema.ts        → Work groups (task teams)
    ├── work-group-employee.schema.ts → Group memberships
    └── work-report.schema.ts       → Work progress reports
```

Database Schemas

WorkEntry (= DPR Entry)

```
{
  date: Date,                    // Report date (required)
  project: ObjectId → Project,   // Site project (required)
  description: string,           // Shift work summary (required)
  employees: ObjectId[] → Employee[], // Workers involved
  createdBy: ObjectId → User,    // Who submitted
  entryType: 'work' | 'department', // Default: 'work'
  organizationId: ObjectId,      // Tenant isolation
  // timestamps: createdAt, updatedAt (auto)
}
// Indexes: date, project, createdBy, (date,-1 + project), (entryType + date,-1)
```

WorkGroup (= Task Group)

```
{
  name: string,                  // Unique group name (required)
  project: ObjectId → Project,   // Associated project (required)
  description: string,           // Group description (required)
  createdBy: ObjectId → User,
  status: 'active' | 'completed', // Default: 'active'
  startDate: Date,               // Default: Date.now
  organizationId: ObjectId,
}
```

WorkReport (= Progress Report)

```
{
  group: ObjectId → WorkGroup,   // Related work group (required)
  workDate: Date,                // Date of work (required)
  description: string,           // Work description (required)
  project: ObjectId → Project,
  employees: ObjectId[] → Employee[],
  createdBy: ObjectId → User,
  organizationId: ObjectId,
}
```

API Endpoints (Controller: `/api/v1/work-management`)

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/entries`	createEntry	Submit a new DPR / work entry
GET	`/entries`	findAllEntries	List all entries (filterable by date, project)
PATCH	`/entries/:id`	updateEntry	Update a DPR entry
DELETE	`/entries/:id`	deleteEntry	Delete a DPR entry
POST	`/groups`	createGroup	Create a work task group
GET	`/groups`	findAllGroups	List work groups
GET	`/groups/:id`	findGroupById	Get group details
PATCH	`/groups/:id`	updateGroup	Update group
DELETE	`/groups/:id`	deleteGroup	Delete group
POST	`/groups/:id/employees`	addEmployee	Add employee to group
DELETE	`/groups/:id/employees/:employeeId`	removeEmployee	Remove from group
POST	`/reports`	createReport	Create work progress report
GET	`/reports`	findAllReports	List progress reports

Frontend Service (`frontend/src/services/index.ts`)

```
export const workEntryService = {
  create: (data) => api.post('/work-management/entries', data),
  getAll: (params?) => api.get('/work-management/entries', { params }),
  update: (id, data) => api.patch(`/work-management/entries/${id}`, data),
  delete: (id) => api.delete(`/work-management/entries/${id}`),
};

export const workManagementService = {
  createGroup: (data) => api.post('/work-management/groups', data),
  getAllGroups: (params?) => api.get('/work-management/groups', { params }),
  getGroupById: (id) => api.get(`/work-management/groups/${id}`),
  updateGroup: (id, data) => api.patch(`/work-management/groups/${id}`, data),
  deleteGroup: (id) => api.delete(`/work-management/groups/${id}`),
  addEmployee: (id, employeeId) => api.post(`/work-management/groups/${id}/employees`, { employeeId }),
  removeEmployee: (id, employeeId) => api.delete(`/work-management/groups/${id}/employees/${employeeId}`),
  createReport: (data) => api.post('/work-management/reports', data),
  getAllReports: (params?) => api.get('/work-management/reports', { params }),
};
```

Frontend Page

```
frontend/src/app/(dashboard)/work-management/
├── page.tsx          → Main DPR list page (35KB — tabs, filters, entry cards)
├── new/              → Create new entry page
├── entries/          → Entry detail views
├── detail/           → Group/report detail views
└── department/       → Department-type entries
```

WHAT NEEDS TO BE BUILT/COMPLETED

The reference app ("DPR & Progress") expects:

1. **Weather field** — The current `WorkEntry` schema does NOT have a `weather` field. Add `weather: string` (optional) to the schema.
2. **"Submit Site DPR" modal** — A modal form with: Site Project (dropdown), Report Date (date picker), Weather (text), Shift Work Summary (textarea).
3. **DPR list view** — Card/table showing submitted DPRs with project name, date, weather, summary.
4. **Empty state** — Camera icon + "No DPRs Logged Yet" message when no entries exist.

IMPLEMENTATION INSTRUCTIONS

Backend changes needed:

1. Add `weather?: string` prop to `WorkEntry` schema in `backend/src/modules/work-management/schemas/work-entry.schema.ts`
2. Add `weather` field to `CreateWorkEntryDto` and `UpdateWorkEntryDto`
3. The `workEntryService.create()` and `.update()` already handle generic fields, so service changes should be minimal

Frontend changes needed:

1. The page at `frontend/src/app/(dashboard)/work-management/page.tsx` is the main page (35KB). Update it to match the "DPR & Progress" layout from the reference app.
2. Add a "Submit Site DPR" modal with the form fields shown in the screenshot.
3. Rename the page title to "Daily Progress Reports (DPR)" with subtitle "Site execution milestones, task completion percentages, and material consumption."

MODULE 2: SPEND & VENDORS (Site Spend & Vendor Credit Control)

Reference App UI

- **Sidebar:** "Spend & Vendors"

- **Page Title:** "Site Spend & Vendor Credit Control"
- **Subtitle:** "Material receipts, credit purchases, approval workflows, and Accounts Payable ledger."
- **Tabs:** Expenses Log (count) | Vendor Payables & Credit (count)
- **Primary Action:** "+ Log Site Expense" button (top-right, blue)
- **Tab 1 - Expenses Log:** List of expense records. Empty state: "No expenses recorded yet."
- **Tab 2 - Vendor Payables & Credit:** Vendor credit management

Maps To (This Project's Codebase)

This module combines TWO existing backend modules:

1. **Expenses** (`/api/v1/expenses`) → Tab 1: Expenses Log
2. **Credit Logs** (`/api/v1/credit-logs`) → Tab 2: Vendor Payables & Credit

Backend Module 1: `expenses`

```
backend/src/modules/expenses/
├── expenses.controller.ts      → API endpoints
├── expenses.service.ts         → Business logic (3.2KB)
├── expenses.module.ts
├── dto/
│   ├── create-expense.dto.ts
│   ├── update-expense.dto.ts
│   ├── approve-expense.dto.ts
│   └── index.ts
└── schemas/
    └── expense.schema.ts
```

Expense Schema:

```
{
  title: string,                    // Expense title (required)
  description?: string,             // Details
  items: [{ name: string, amount: number }], // Line items
  amount: number,                   // Total amount (required)
  category: 'materials' | 'transportation' | 'equipment' | 'labor' | 'miscellaneous', // Required
  status: 'pending' | 'approved' | 'rejected', // Default: pending
  date: Date,                       // Expense date (required)
  project: ObjectId → Project,      // Site project (required)
  receiptUrl?: string,              // Receipt attachment URL
  submittedBy: ObjectId → User,     // Who logged it (required)
  approvedBy?: ObjectId → User,     // Who approved it
  organizationId: ObjectId,
}
// Indexes: (project + date,-1), (project + status), submittedBy, (status + date,-1), (project + category)
```

Expenses API Endpoints (Controller: `/api/v1/expenses`):

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/`	create	Log a new site expense
GET	`/`	findAll	List expenses (filterable)
GET	`/my-expenses`	findMyExpenses	Get current user's expenses
GET	`/:id`	findOne	Get expense details
PATCH	`/:id`	update	Update expense
PATCH	`/:id/approve`	approve	Approve/reject expense
DELETE	`/:id`	remove	Delete expense

Backend Module 2: `credit-logs`

```
backend/src/modules/credit-logs/
├── credit-logs.controller.ts   → API endpoints
├── credit-logs.service.ts      → Business logic (16KB)
├── credit-logs.module.ts
├── dto/
│   ├── create-credit-log.dto.ts
│   ├── update-credit-log.dto.ts
│   ├── query-credit-log.dto.ts
│   ├── add-credit-log-expense.dto.ts
│   └── index.ts
└── schemas/
    └── credit-log.schema.ts
```

CreditLog Schema (= Vendor Credit / Advance):

```
{
  transactionDate: Date,              // Default: now
  creditedBy: ObjectId → User,        // Who gave the credit (required)
  amount: number,                     // Credit amount (required, min: 0)
  description: string,                // Purpose (required)
  project?: ObjectId → Project,       // Associated project
  employee?: ObjectId → Employee,     // Associated employee (if worker advance)
  category?: string,                  // Credit category
  individualName?: string,            // Vendor/individual name
  paymentMethod: 'cash' | 'bank_transfer' | 'upi' | 'cheque' | 'other', // Default: cash
  expenses: [{                        // Sub-expenses against this credit
    amount: number,
    description: string,
    date: Date,
    recordedBy?: ObjectId → User,
    transactionId?: string,
  }],
  amountSpent: number,                // Auto-calculated (sum of expenses)
  remainingBalance: number,           // Auto-calculated (amount - amountSpent)
  lastExpenseDate?: Date,             // Auto-updated
  status: 'active' | 'fully_utilized' | 'closed', // Auto-calculated
  organizationId: ObjectId,
}
// Pre-save hook: recalculates amountSpent, remainingBalance, status on save
```

Credit Logs API Endpoints (Controller: `/api/v1/credit-logs`):

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/`	create	Create new credit entry
GET	`/`	findAll	List credit logs (filterable)
GET	`/summary`	getSummary	Get credit summary statistics
GET	`/:id`	findOne	Get credit log details
PATCH	`/:id`	update	Update credit log
DELETE	`/:id`	remove	Delete credit log
POST	`/:id/expenses`	addExpense	Add expense against credit
DELETE	`/:id/expenses/:expenseId`	removeExpense	Remove expense from credit

Frontend Services

```
// Expenses
export const expenseService = {
  getAll: (params?) => api.get('/expenses', { params }),
  getById: (id) => api.get(`/expenses/${id}`),
  create: (data) => api.post('/expenses', data),
  update: (id, data) => api.patch(`/expenses/${id}`, data),
  approve: (id, data) => api.patch(`/expenses/${id}/approve`, data),
  delete: (id) => api.delete(`/expenses/${id}`),
};

// Credit Logs
export const creditLogService = {
  getAll: (params?) => api.get('/credit-logs', { params }),
  getById: (id) => api.get(`/credit-logs/${id}`),
  create: (data) => api.post('/credit-logs', data),
  update: (id, data) => api.patch(`/credit-logs/${id}`, data),
  delete: (id) => api.delete(`/credit-logs/${id}`),
  addExpense: (id, data) => api.post(`/credit-logs/${id}/expenses`, data),
  removeExpense: (id, expenseId) => api.delete(`/credit-logs/${id}/expenses/${expenseId}`),
  getSummary: (params?) => api.get('/credit-logs/summary', { params }),
};
```

Frontend Pages

```
frontend/src/app/(dashboard)/expense/
├── page.tsx              → Main expenses page (29KB, existing)
├── ExpenseReportTab.tsx  → Expense report tab component
└── new/                  → Create new expense page

frontend/src/app/(dashboard)/credit-logs/
└── page.tsx              → Credit logs page (80KB, existing — very large)
```

WHAT NEEDS TO BE BUILT/COMPLETED

The reference app combines expenses + vendor credit into ONE page with TWO tabs:

1. **Tab 1 "Expenses Log"** — Use existing `expenseService` API. Show expense cards with title, amount, category, status, date, project. "+ Log Site Expense" creates a new expense.
2. **Tab 2 "Vendor Payables & Credit"** — Use existing `creditLogService` API. Show vendor credit entries with remaining balance, expenses tracked. Support "Add Expense" against a credit line.

IMPLEMENTATION INSTRUCTIONS

No backend changes needed — both modules are fully implemented.

Frontend: Create a combined page (or update the existing `/expense/page.tsx`) that:

- Uses `expenseService.getAll()` for Tab 1 data
- Uses `creditLogService.getAll()` for Tab 2 data
- Has the "Log Site Expense" modal calling `expenseService.create()`
- Tab counts show `(${expenses.length})` and `(${creditLogs.length})`
- Rename sidebar nav entry if desired (currently "Payments" pointing to `/expense`)

MODULE 3: PAYROLL (Payroll Engine & Worker Advances)

Reference App UI

- **Sidebar:** "Payroll"

- **Page Title:** "Payroll Engine & Worker Advances"
- **Subtitle:** "Weekly wage calculation, FIFO advance deductions, payslips, and journal postings."
- **Tabs:** Payroll Runs (count) | Worker Advances (count)
- **Action Buttons:** "$ Record Advance" (secondary) | "+ Calculate Weekly Payroll" (primary, blue)
- **Tab 1 - Payroll Runs:** List of payroll calculation results. Empty state: "No Payroll Runs Generated Yet" + "Click 'Calculate Weekly Payroll' to run attendance calculations for a project."
- **Tab 2 - Worker Advances:** Employee advance/withdrawal records
- **Calculate Weekly Payroll Modal:**
  - Site Project * (dropdown: Select Project)
  - Period Start (e.g. Monday) * (date picker DD/MM/YYYY)
  - Period End (e.g. Sunday) * (date picker DD/MM/YYYY)
  - Note: "Payroll calculations require that site attendance for the selected date range has been locked."
  - Buttons: Cancel | Calculate Vouchers

Maps To (This Project's Codebase)

Backend Module: `payments`

```
backend/src/modules/payments/
├── payments.controller.ts      → API endpoints
├── payments.service.ts         → Business logic (33KB — very large, complex)
├── payments.module.ts
├── dto/
│   ├── create-payment.dto.ts
│   ├── query-payment.dto.ts
│   ├── batch-create-payment.dto.ts
│   └── index.ts
└── schemas/
    └── payment.schema.ts
```

Payment Schema:

```
{
  employee: ObjectId → Employee,       // Worker (required)
  project?: ObjectId → Project,        // Site project
  type: 'salary' | 'advance' | 'withdrawal', // Default: salary
  baseSalary: number,                  // Gross calculated salary (default: 0)
  expenses: number,                    // Deductions (default: 0)
  amount: number,                      // Net payable = baseSalary − expenses (required, min: 0)
  method: 'cash' | 'bank_transfer' | 'upi' | 'cheque', // Default: cash
  status: 'pending' | 'approved' | 'paid' | 'rejected' | 'cancelled', // Default: pending
  dateRange?: { from: Date, to: Date }, // Payroll period
  description?: string,                // Notes (max 500 chars)
  approvedBy?: ObjectId → User,
  approvedByAdminName?: string,
  approvedAt?: Date,
  approvalTimestamp?: Date,
  processedBy?: ObjectId → User,
  processedAt?: Date,
  notes?: string,
  createdBy: ObjectId → User,
  organizationId: ObjectId,
}
```

Payment Enums (from constants):

```
enum PaymentType { SALARY = 'salary', ADVANCE = 'advance', WITHDRAWAL = 'withdrawal' }
enum PaymentMethod { CASH = 'cash', BANK_TRANSFER = 'bank_transfer', UPI = 'upi', CHEQUE = 'cheque' }
enum PaymentStatus { PENDING = 'pending', APPROVED = 'approved', PAID = 'paid', REJECTED = 'rejected', CANCELLED = 'cancelled' }
```

Payments API Endpoints (Controller: `/api/v1/payments`):

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/`	create	Create single payment/voucher
POST	`/batch`	batchCreate	Create multiple payments at once (bulk payroll)
GET	`/`	findAll	List payments (filterable by employee, project, status, type, date)
GET	`/summary`	getSummary	Get payment summary stats (total paid, pending, etc.)
GET	`/calculate-salary`	calculateSalary	**KEY:** Calculate salary for employee for date range (uses attendance data)
GET	`/remaining-salary`	getRemainingSalary	Get remaining salary due for employees
GET	`/:id`	findOne	Get payment details
PATCH	`/:id/approve`	approve	Approve payment
PATCH	`/:id/reject`	reject	Reject payment
PATCH	`/:id/process`	process	Mark payment as processed/paid
DELETE	`/:id`	remove	Delete payment

Also Uses: `credit-logs` module for Worker Advances

- `type: 'advance'` payments in the payments collection = Worker Advances
- Credit logs track the running balance of advances given to employees

Frontend Service

```
export const paymentService = {
  getAll: (params?) => api.get('/payments', { params }),
  getById: (id) => api.get(`/payments/${id}`),
  create: (data) => api.post('/payments', data),
  batchCreate: (data) => api.post('/payments/batch', data),
  approve: (id) => api.patch(`/payments/${id}/approve`),
  reject: (id) => api.patch(`/payments/${id}/reject`),
  process: (id) => api.patch(`/payments/${id}/process`),
  getSummary: (params?) => api.get('/payments/summary', { params }),
  calculateSalary: (employeeId, from, to) =>
    api.get('/payments/calculate-salary', { params: { employeeId, from, to } }),
  getRemainingSalary: (params?) => api.get('/payments/remaining-salary', { params }),
  delete: (id) => api.delete(`/payments/${id}`),
};
```

Frontend Page

```
frontend/src/app/(dashboard)/payment-central/
├── page.tsx    → Main payment central page (69KB — very large, complex)
└── new/        → Create new payment page
```

WHAT NEEDS TO BE BUILT/COMPLETED

The reference app "Payroll" expects:

1. **Tab 1 "Payroll Runs"** — Shows weekly payroll calculation results (batch payment vouchers). The "Calculate Weekly Payroll" button should:
   - Open a modal with: Project dropdown, Period Start date, Period End date
   - Call `paymentService.calculateSalary()` for each employee in the project for the date range
   - Then call `paymentService.batchCreate()` to create all salary vouchers at once
   - Display results as a list of payroll runs grouped by week + project
2. **Tab 2 "Worker Advances"** — Filter payments where `type === 'advance'`. The "Record Advance" button creates a payment with `type: 'advance'`.

3. **FIFO advance deductions** — When calculating payroll, automatically deduct outstanding advances from salary. This logic may need to be in `payments.service.ts`.

IMPLEMENTATION INSTRUCTIONS

Backend: The `/payments/calculate-salary` endpoint already exists and calculates salary from attendance data. You may need to:

1. Add a `/payments/payroll-run` endpoint that calculates salaries for ALL employees on a project for a date range and creates batch payment records
2. Add advance deduction logic (FIFO) in the payroll calculation
3. Potentially add a `payrollRunId` or `batchId` field to group payments by payroll run

Frontend: Update `payment-central/page.tsx` (69KB) to match the "Payroll Engine" UI with two tabs and the Calculate Weekly Payroll modal.

MODULE 4: FINANCIAL LEDGER

Reference App UI

- **Sidebar:** "Financial Ledger"
- **Purpose:** Central financial transaction log — all money in/out across the organization

Maps To (This Project's Codebase)

This module uses THREE existing backend modules:

1. **Transactions** (`/api/v1/transactions`) → Primary ledger
2. **Running Costs** (`/api/v1/running-costs`) → Recurring site costs
3. **Credit Logs** (`/api/v1/credit-logs`) → Credit tracking (shared with Spend & Vendors)

Backend Module: `transactions`

```
backend/src/modules/transactions/
├── transactions.controller.ts  → API endpoints
├── transactions.service.ts     → Business logic
├── transactions.module.ts
├── dto/
│   ├── create-transaction.dto.ts
│   ├── update-transaction.dto.ts
│   ├── query-transaction.dto.ts
│   └── index.ts
└── schemas/
    └── transaction.schema.ts
```

Transaction Schema:

```
{
  transactionId: string,              // Unique ID (required, auto-generated)
  date: Date,                         // Transaction date (default: now)
  type: 'incoming' | 'outgoing',      // Money direction (required)
  category: 'salary' | 'vendor' | 'client' | 'expense_reimbursement' | 'site_cost' | 'other',
  payerName: string,                  // Who paid (required)
  payeeName: string,                  // Who received (required)
  amount: number,                     // Amount (required, min: 0)
  method: 'cash' | 'bank_transfer' | 'upi' | 'card' | 'cheque' | 'other',
  status: 'pending' | 'completed' | 'failed' | 'refunded',
  description?: string,
  recordedBy?: ObjectId → User,
  metadata: {},                       // Flexible key-value storage
  organizationId: ObjectId,
}
// Indexes: date,-1, status, type, text search (payerName + payeeName + description)
```

Transactions API Endpoints (Controller: `/api/v1/transactions`):

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/`	create	Record new transaction
GET	`/`	findAll	List transactions (filterable)
GET	`/summary`	getSummary	Get financial summary
GET	`/:id`	findOne	Get transaction details
PATCH	`/:id`	update	Update transaction
DELETE	`/:id`	remove	Delete (super_admin only)

Backend Module: `running-costs`

```
backend/src/modules/running-costs/
├── running-costs.controller.ts
├── running-costs.service.ts
├── running-costs.module.ts
├── schemas/
│   └── running-cost.schema.ts
```

RunningCost Schema:

```
{
  date: Date,                    // Cost date (required)
  project: ObjectId → Project,   // Site project (required)
  amount: number,                // Cost amount (required)
  description: string,           // What the cost is for (required)
  enteredBy: ObjectId → User,    // Who logged it (required)
  userName: string,              // Denormalized user name
  userRole: string,              // Denormalized user role
  paymentMethod: 'cash' | 'bank_transfer' | 'upi' | 'cheque' | 'other',
  organizationId: ObjectId,
}
```

Running Costs API (Controller: `/api/v1/running-costs`):

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/`	create	Log new running cost
GET	`/`	findAll	List running costs (by project, date range)
PATCH	`/:id`	update	Update cost
DELETE	`/:id`	remove	Delete cost

Frontend Services

```
export const transactionService = {
  getAll: (params?) => api.get('/transactions', { params }),
  getById: (id) => api.get(`/transactions/${id}`),
  getSummary: (params?) => api.get('/transactions/summary', { params }),
  create: (data) => api.post('/transactions', data),
  update: (id, data) => api.patch(`/transactions/${id}`, data),
  delete: (id) => api.delete(`/transactions/${id}`),
};

export const runningCostService = {
  create: (data) => api.post('/running-costs', data),
  getAll: (params) => api.get('/running-costs', { params }),
  update: (id, data) => api.patch(`/running-costs/${id}`, data),
  delete: (id) => api.delete(`/running-costs/${id}`),
};
```

Frontend Pages

```
frontend/src/app/(dashboard)/running-cost/
├── page.tsx          → Running cost page (89KB — very large)
├── components/       → Sub-components
└── entries/          → Entry detail views

frontend/src/app/(dashboard)/credit-logs/
└── page.tsx          → Credit logs page (80KB)
```

WHAT NEEDS TO BE BUILT/COMPLETED

The reference app "Financial Ledger" expects a unified financial view showing:

1. **All incoming transactions** (client payments, funds received)
2. **All outgoing transactions** (salaries, vendor payments, running costs, expenses)
3. **Running balance** — cumulative in/out with current balance
4. **Filters** — by date range, type (in/out), category, project
5. **Integration** — automatically create transaction records when payments, expenses, or running costs are recorded

IMPLEMENTATION INSTRUCTIONS

Backend: Transactions module is already implemented. May need:

1. Auto-create `Transaction` records when: a Payment is marked `paid`, an Expense is `approved`, a RunningCost is created
2. Add summary aggregation for overall financial health (total in, total out, balance)

Frontend: Create a new `/financial-ledger/page.tsx` (or repurpose running-cost page) showing:

- Table/list of all transactions with type badge (incoming=green, outgoing=red)
- Summary cards: Total Income, Total Expenses, Net Balance
- Filters: date range, type, category, project

MODULE 5: REPORTS & EXPORTS

Reference App UI

- **Sidebar:** "Reports & Exports"
- **Purpose:** Generate, schedule, download, and email reports in PDF/Excel format

Maps To (This Project's Codebase)

This module uses TWO existing backend modules:

1. **Reports** (`/api/v1/reports`) → On-demand report generation
2. **Report Automation** (`/api/v1/report-automation`) → Scheduled reports + BullMQ workers

Backend Module 1: `reports`

```
backend/src/modules/reports/
├── reports.controller.ts       → API endpoints (admin + employee self-service)
├── reports.service.ts          → Report generation logic (46KB — VERY large)
├── reports.module.ts
├── dto/
│   ├── generate-report.dto.ts  → Report request parameters
│   ├── query-report.dto.ts
│   └── index.ts
└── schemas/
    └── report.schema.ts
```

Report Schema:

```
{
  title: string,                      // Report title (required)
  type: ReportType,                   // One of 30+ report types (required)
  filters: {                          // Report parameters
    employeeId?: ObjectId,
    projectId?: ObjectId,
    groupId?: ObjectId,
    dateFrom?: Date,
    dateTo?: Date,
  },
  generatedBy: ObjectId → User,       // Who requested it
  fileUrl?: string,                   // S3 URL for downloadable file
  status: 'queued' | 'generating' | 'ready' | 'failed' | 'expired',
  data: any,                          // Report data (Mixed/JSON)
  organizationId: ObjectId,
}
```

All 30+ Report Types (from `backend/src/common/constants/index.ts`):

```
enum ReportType {
  // Workforce Reports
  EARNINGS = 'earnings',
  ATTENDANCE = 'attendance',
  EMPLOYEE_WISE = 'employee_wise',
  GROUP_JOINT = 'group_joint',
  GROUP_SEPARATE = 'group_separate',
  CREDIT_SALARY = 'credit_salary',
  // Project Reports
  PROJECT_SUMMARY = 'project_summary',
  SITE_PROGRESS = 'site_progress',
  WORK_MANAGEMENT = 'work_management',
  DEPARTMENT = 'department',
  // Financial Reports
  PAYMENT_STATUS = 'payment_status',
  PAYMENT_HISTORY = 'payment_history',
  FINANCIAL_AUDIT = 'financial_audit',
  EARNING_SUMMARY = 'earning_summary',
  FUNDS_RECEIVED = 'funds_received',
  // Cost Control Reports
  PROJECT_COSTS = 'project_costs',
  TRANSPORTATION_COSTS = 'transportation_costs',
  MATERIAL_USAGE = 'material_usage',
  EXPENSE_SUMMARY = 'expense_summary',
  UPAD = 'upad',
  // Weekly Reports
  WEEKLY_SITE = 'weekly_site',
  WEEKLY_COST_ANALYSIS = 'weekly_cost_analysis',
  WEEKLY_LABOUR = 'weekly_labour',
  // Monthly Reports
  WAGE_REGISTER = 'wage_register',
  EMPLOYEE_LEDGER = 'employee_ledger',
  MEASUREMENT_SHEET = 'measurement_sheet',
  ABSTRACT_SHEET = 'abstract_sheet',
  CONTRACTOR_BILLING = 'contractor_billing',
  SITE_COST_ANALYSIS = 'site_cost_analysis',
  COMPLETE_FINANCIAL_AUDIT = 'complete_financial_audit',
  PROJECT_PROFITABILITY = 'project_profitability',
  RESOURCE_UTILIZATION = 'resource_utilization',
  // Admin Activity Reports
  ADMIN_ACTIVITY = 'admin_activity',
  REPORT_ACCESS_AUDIT = 'report_access_audit',
}
```

Reports API Endpoints (Controller: `/api/v1/reports`):

METHOD	PATH	HANDLER	DESCRIPTION
POST	`/generate`	generate	Generate a new report (admin)
GET	`/`	findAll	List generated reports (admin)
GET	`/:id`	findOne	Get report details
GET	`/:id/download`	download	Download report as Excel file
PATCH	`/:id`	update	Update report
DELETE	`/:id`	remove	Delete report
POST	`/my-report`	generateMyReport	Generate self-service report (employee)
GET	`/my-reports`	findMyReports	List employee's own reports
GET	`/my-reports/:id/download`	downloadMyReport	Download own report

Backend Module 2: `report-automation`

```
backend/src/modules/report-automation/
├── report-automation.controller.ts → API endpoints (15KB)
├── report-automation.module.ts     → Module with BullMQ queue registration (5.7KB)
├── controllers/                    → Sub-controllers
├── services/                       → Report generation services
├── workers/                        → BullMQ queue workers (process reports async)
├── dto/
└── schemas/
    ├── report-execution.schema.ts  → Track report job status
    ├── admin-activity-log.schema.ts → Admin action tracking
    └── report-access-log.schema.ts  → Who viewed/downloaded reports
```

7 BullMQ Report Queues:

```
enum ReportQueue {
  ATTENDANCE = 'report_attendance',
  WORKFORCE = 'report_workforce',
  FINANCE = 'report_finance',
  COST_ANALYSIS = 'report_cost_analysis',
  SITE_MANAGEMENT = 'report_site_management',
  PROJECT_ANALYTICS = 'report_project_analytics',
  AUDIT = 'report_audit',
}
```

Report Formats:

```
enum ReportFormat { EXCEL = 'excel', PDF = 'pdf', BOTH = 'both' }
enum ReportFrequency { DAILY = 'daily', WEEKLY = 'weekly', MONTHLY = 'monthly', ON_DEMAND = 'on_demand' }
```

Frontend Service

```
export const reportService = {
  getAll: (params?) => api.get('/reports', { params }),
  getById: (id) => api.get(`/reports/${id}`),
  generate: (data) => api.post('/reports/generate', data),
  download: (id) => api.get(`/reports/${id}/download`, { responseType: 'blob' }),
  update: (id, data) => api.patch(`/reports/${id}`, data),
  delete: (id) => api.delete(`/reports/${id}`),
  generateMyReport: (data) => api.post('/reports/my-report', data),
  getMyReports: (params?) => api.get('/reports/my-reports', { params }),
  downloadMyReport: (id) => api.get(`/reports/my-reports/${id}/download`, { responseType: 'blob' }),
};
```

Frontend Pages

```
frontend/src/app/(dashboard)/reports/
└── page.tsx              → On-demand reports page (72KB)

frontend/src/app/(dashboard)/report-center/
├── page.tsx              → Report center with automation (58KB)
└── monitoring/           → Report job monitoring
```

WHAT NEEDS TO BE BUILT/COMPLETED

The reference app "Reports & Exports" expects:

1. **Report catalog** — Grid/cards of available report types grouped by category (Workforce, Project, Financial, Cost Control, Weekly, Monthly)
2. **Generate flow** — Select report type → configure filters (project, employee, date range) → generate
3. **Download center** — List of generated reports with status (queued → generating → ready) and download buttons (PDF/Excel)
4. **Scheduled reports** — Configure automated report delivery (daily digest at 9:30 PM IST)
5. **Email delivery** — Send report to admin users via SMTP

IMPLEMENTATION INSTRUCTIONS

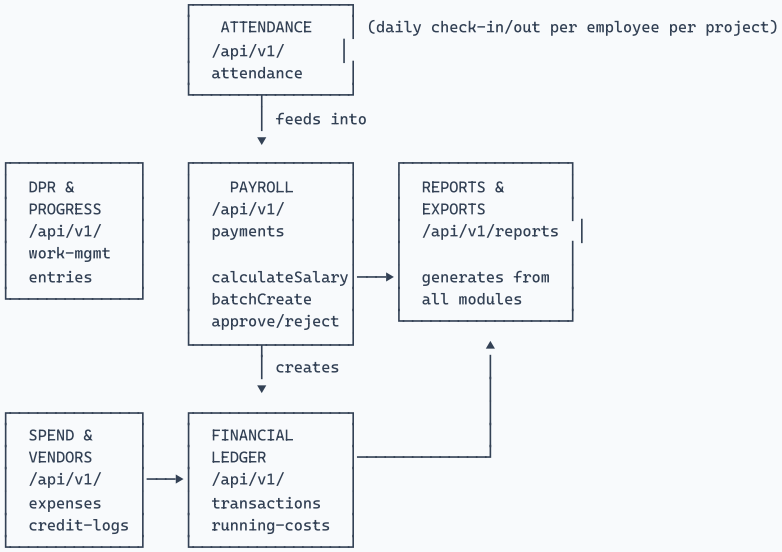
Backend: Already fully implemented with 30+ report types and BullMQ automation.

Frontend: The pages exist (72KB + 58KB) but may need UI updates to match the reference app's layout. Ensure:

1. Report type selector with category grouping
2. Filter configuration form (project dropdown, employee dropdown, date range picker)
3. Download status badges with progress indicators
4. Combine `/reports` and `/report-center` into a single "Reports & Exports" page if the reference app has one page

---

CROSS-MODULE DATA FLOW



Key Business Rules

1. **Payroll depends on Attendance** — `calculateSalary` aggregates attendance records for the date range × employee × project to compute gross wages
2. **Advances deduct from Payroll** — Payments with `type: 'advance'` should be tracked and deducted FIFO from future salary calculations
3. **Expenses create Transactions** — When an expense is approved, a corresponding outgoing Transaction should be created in the Financial Ledger
4. **Reports aggregate everything** — The 30+ report types query from attendance, payments, expenses, running costs, credit logs, and transactions collections
5. **All data is tenant-isolated** — Every query is automatically filtered by `organizationId` via the Mongoose plugin

---

SIDEBAR NAME MAPPING (Quick Reference)

REFERENCE APP SIDEBAR	THIS PROJECT'S SIDEBAR	ROUTE	BACKEND MODULE(S)
Dashboard	Dashboard	`/`	dashboard
Projects	Projects	`/projects`	projects
Employees	Employees	`/employees`	employees
Attendance	Attendance	`/attendance`	attendance

REFERENCE APP SIDEBAR	THIS PROJECT'S SIDEBAR	ROUTE	BACKEND MODULE(S)
DPR & Progress	**Work Management**	`/work-management`	**work-management**
Spend & Vendors	**Payments (Expenses)**	`/expense`	**expenses + credit-logs**
Payroll	**Payment Central**	`/payment-central`	**payments**
Financial Ledger	**Running Cost + Credit Logs**	`/running-cost` , `/credit-logs`	**transactions + running-costs + credit-logs**
Reports & Exports	**Reports + Report Center**	`/reports` , `/report-center`	**reports + report-automation**
Settings	Settings	`/settings`	settings